There is an enum called 'Vehicle' whose variants include 'Bicycle' and 'Car'. We are creating a couple of Vehicle variables in *main()* and printing them using a helper function called *print_vehicle*. There is nothing surprising about the output:

```
A car with reg. no.: KL 00 X 0000.
A bicycle.
A car with reg. no.: KL 00 Y 1111.
```

As you can guess, the match construct used inside *print_vehicle()* is the Rust counterpart of C's switch (simply speaking). However, what confuses a C programmer is the fact that the variant Car has some data associated with it.

Stop focusing on the code for a while and think about what the program does. It stores and displays vehicle objects, of which cars can (in fact, must) have a registration number and bicycles cannot. How would you code this in C? You'll have a struct named Vehicle that includes all the parameters related to all kinds of vehicles and set or retrieve them based on the kind of the vehicle stored in a field called, say, 'type'. What is the problem here? There is nothing that prevents you from accessing the (invalid) data stored in the *regno* field of a Vehicle object even if its type is set to Bicycle:

```
typedef enum {
  BICYCLE,
  CAR,
  ...
} VehicleType;

typedef struct {
  VehicleType type;
  char regno[MAX_REGNO];
  ...
} Vehicle;

// Somewhere in the middle, where
vehicle.type can be BICYCLE:
add_to_serviced_vehicles(vehicle.regno);
// no compile-time error
```

What about unions? Yes, unions let us have some separation, but it's still up to you to decide how to interpret that data. But this is exactly what Rust does here on its own. The enum we saw in the Rust code is technically called a tagged union, something that lets multiple kinds of data coexist, settable or gettable only in accordance with the kind (tag) of object. We need to match this data to extract their values, making it impossible to have incorrect interpretations. This comparison is summarised in Table 1.

Tip: In the Rust code we saw, Bicycle is a simple variant while Car is a tuple variant. There is one more kind called struct variant, in which you can label the fields like in a struct.

Option type

You've probably just started liking the concept of tagged unions and would like to try it out sometime in the future. Guess what — you cannot avoid them even if you wanted to! One tagged union called *Option<T>*, defined by the *std::option* module, is heavily used by the Rust library.

What is an option type? It is the return type of functions that cannot guarantee there will always be a valid value to return. A generic example is a function that searches for a substring in a string, returning the position of the occurrence. Another example is a function that pops the top element off a stack. In the first example, what if no such substring was there? In the second example, what if the stack is already empty?

The C-style workarounds include returning special constants ('magic numbers') like -1, or even aborting when called without sufficient checks like stack underflow. What about the option type? *Option<T>* (where T can be any type like i32 or string) has two variants, *Some<T>* and *None*. *Some<T>* contains a value of type T. *None* means there is no value.

Let's see a simple example that shows *Option<T>* in action:

```
fn main() {
  let haystack = "the quick brown fox";

  for needle in ["fox", "rabbit",
"brown"] {
    let pos = haystack.find(needle);

    match pos {
      Some(pos) => println!("'{}' found
at: {}", needle, pos),
      None => println!("'{}' not
found.", needle),
    }
  }
}
```

As you might've guessed, the output is:

```
'fox' found at: 16
'rabbit' not found.
'brown' found at: 10
```

Table 1: C workarounds vs proper tagged unions

	enum + struct (C)	enum + union (C)	Tagged union (Rust)
Valid tag at a time	Only one	Only one	Only one
Valid fields at a time	Related to any one variant	Related to any one variant	Related to the current variant
Storage	Sum of all variants	Maximum of all variants	Maximum of all variants
Incorrect access	Possible	Possible	Not possible
Data when misinterpreted	Junk	Junk	Not allowed